DAAR – Projet 1

Clone de `egrep` avec support partiel des ERE

ALAOUCHICHE Abderrahmane Yaakoub KOUACHE Riad

Sorbonne Université – Master RES – 31 octobre 2026

Table des matières

1 Introduction **1**

2 Recherche par automate **2**
2.1 Arbre syntaxique 2
2.2 AFN avec ε-transitions (Aho-Ullman) 2
2.3 AFD par la méthode des sous-ensembles 2
2.4 Minimisation 3
2.5 Recherche dans une ligne 3

3 Recherche d'un motif littéral : KMP **4**

4 Analyse théorique et discussion **4**

5 Tests de correction **5**
5.1 Instances de test 5
5.2 Méthode et résultats 5

6 Tests de performance **6**
6.1 Protocole 6
6.2 Résultats 6

7 Conclusion et perspectives **7**

1 Introduction

L'objectif du projet est d'écrire un clone de la commande `egrep` : étant donnés une expression régulière R et un fichier texte de lignes $L_1, \ldots, L_l$, le programme affiche, dans l'ordre, chaque ligne L_i qui contient au moins une sous-chaîne appartenant au langage $\mathcal{L}(R)$. Les lignes sont traitées indépendamment ; le problème se réduit donc à l recherches du même motif.

Conformément à l'énoncé, nous nous restreignons au sous-ensemble suivant de la norme ERE [6] (tableau 1). Les autres éléments de la norme (`+`, `?`, classes de caractères, ancres, échappement par `\`) ne sont pas pris en charge.

Le programme (`src/egrep_clone.py`, Python 3) suit les deux stratégies proposées en cours :

— si le motif contient un métacaractère (`( ) | * .`), il est transformé en arbre syntaxique, puis en automate fini non déterministe (AFN) avec ε-transitions selon la méthode d'Aho-Ullman, puis en automate fini déterministe (AFD) par la méthode des sous-ensembles, puis en AFD minimal, qui est enfin utilisé pour tester chaque ligne (section 2) ;

— si le motif est une simple suite de concaténations de lettres, la recherche utilise l'algorithme de Knuth-Morris-Pratt (KMP) (section 3).

Syntaxe	Opérateur	Langage reconnu
`a`	lettre ASCII	$\{a\}$
`.`	caractère universel	tout caractère
R_1R_2	concaténation	$\mathcal{L}(R_1) \cdot \mathcal{L}(R_2)$
$R_1 \mid R_2$	alternative	$\mathcal{L}(R_1) \cup \mathcal{L}(R_2)$
R`*`	étoile de Kleene	$\mathcal{L}(R)^*$
`(`R`)`	parenthèses	$\mathcal{L}(R)$

TABLE 1 – Opérateurs pris en charge. Priorités décroissantes : étoile, concaténation, alternative.

Comme `egrep`, le programme n'affiche que les lignes trouvées et renvoie le code 0 (au moins une ligne), 1 (aucune) ou 2 (erreur). La section 4 analyse les algorithmes, la section 5 présente les tests de correction et la section 6 les tests de performance.

2 Recherche par automate

2.1 Arbre syntaxique

Le motif est d'abord rendu explicite : la fonction `inserer_concat` insère l'opérateur interne `&` entre deux symboles consécutifs qui se concatènent (le point étant déjà le caractère universel, il ne peut servir d'opérateur de concaténation). La fonction `parse_regex` construit ensuite l'arbre en un seul parcours de gauche à droite, avec deux piles : une pile de sous-arbres et une pile d'opérateurs. Lorsqu'un opérateur arrive, les opérateurs de priorité supérieure ou égale sont dépilés et appliqués aux sous-arbres ; une parenthèse fermante dépile jusqu'à la parenthèse ouvrante. Les nœuds de l'arbre sont des instances de `Carac`, `Concat`, `Alternative` et `Etoile`. Un motif mal formé (parenthèse non fermée, opérande manquant, motif vide) provoque une erreur.

2.2 AFN avec ε-transitions (Aho-Ullman)

La fonction `construire_afn` parcourt l'arbre récursivement et associe à chaque nœud un fragment d'automate ayant un unique état initial et un unique état final [1, 7] :

- `Carac(a)` : deux états reliés par une transition étiquetée a (l'étiquette `.` représente n'importe quel caractère) ;
- `Concat` : une ε-transition relie la fin du fragment gauche au début du fragment droit ;
- `Alternative` : un nouvel état initial mène par ε aux deux fragments, dont les états finals mènent par ε à un nouvel état final ;
- `Etoile` : de nouveaux états initial et final, avec des ε-transitions permettant de sauter le fragment ou de le répéter.

Chaque état (classe `Etat`) stocke un dictionnaire *caractère* $\rightarrow$ *liste d'états* et une liste d'ε-successeurs. Chaque nœud ajoute au plus deux états : l'AFN a donc $O(r)$ états et transitions pour un motif de taille r.

2.3 AFD par la méthode des sous-ensembles

Chaque état de l'AFD est un ensemble d'états de l'AFN (représenté par un `frozenset` et numéroté dans un dictionnaire). L'état initial est la ε-fermeture de l'état initial de l'AFN ; pour un état S et un symbole a, le successeur est la ε-fermeture des états atteints depuis S par a (ou par une transition `.`). Les ε-fermetures sont calculées par un parcours en profondeur (`fermeture_epsilon`). Un état de l'AFD est final s'il contient l'état final de l'AFN.

Deux choix rendent l'AFD directement utilisable pour la recherche :

— **alphabet réduit** : seuls les caractères présents dans le motif sont des symboles distincts ; tous les autres caractères sont regroupés en un symbole unique `AUTRE`, qui n'est accepté que par les transitions étiquetées par le point. L'AFD reste complet et de petite taille, même pour un texte Unicode ;
— **recherche d'une sous-chaîne** : la ε-fermeture de l'état initial de l'AFN est ajoutée à chaque sous-ensemble. L'automate reconnaît alors $\Sigma^* R$, c'est-à-dire tous les mots dont un suffixe appartient à $\mathcal{L}(R)$. Une ligne contient une occurrence de R si et seulement si l'un de ses préfixes est reconnu : un seul parcours de la ligne suffit, sans relancer l'automate à chaque position de départ.

2.4 Minimisation

La fonction `minimiser_afd` applique l'algorithme de raffinement de partitions de Moore [3]. On part de la partition {états finals, états non finals} ; à chaque tour, deux états d'une même classe sont séparés si, pour un symbole, leurs successeurs sont dans des classes différentes. On s'arrête lorsque le nombre de classes ne change plus ; chaque classe devient un état de l'AFD minimal.

La figure 1 montre l'AFD minimal obtenu par le programme pour le motif `S(a|g|r)*on` de l'énoncé. Le tableau 2 donne la taille des automates construits pour les motifs utilisés dans les tests de performance : la minimisation réduit l'AFD lorsque plusieurs sous-ensembles sont équivalents (par exemple de 7 à 4 états pour `S(a|g|r)*on`).

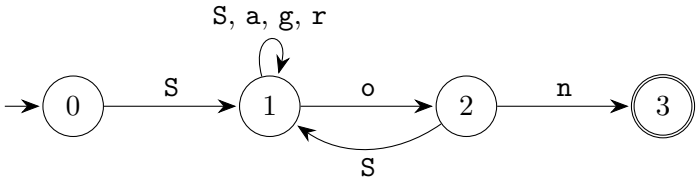


FIGURE 1 – AFD minimal construit pour `S(a|g|r)*on` (alphabet {`S`, `a`, `g`, `n`, `o`, `r`} plus `AUTRE`). Les transitions non représentées mènent à l'état 0, sauf celles sur `S`, qui mènent à l'état 1. La recherche s'arrête dès l'état final 3.

Motif	États AFN	États AFD	États AFD minimal
`end`	6	4	4
`Babylon`	14	8	8
`a.b`	6	6	6
`(the|and)`	14	7	6
`a.*e`	8	5	3
`S(a|g|r)*on`	18	7	4
`B(a|e)*by`	14	6	4

TABLE 2 – Nombre d'états des automates construits (`results/automata_sizes.csv`).

2.5 Recherche dans une ligne

La fonction `correspond_afd` part de l'état initial et lit la ligne caractère par caractère ; chaque caractère est remplacé par `AUTRE` s'il n'appartient pas à l'alphabet du motif, puis une seule transition est suivie. La ligne est acceptée dès qu'un état final est atteint (ou si l'état initial est final, cas des motifs reconnaissant le mot vide comme `a*`, qui correspondent à toutes les lignes, comme avec `egrep`).

3 Recherche d'un motif littéral : KMP

Lorsque le motif P (de longueur m) ne contient aucun métacaractère, il s'agit de rechercher un facteur dans une chaîne. L'algorithme de Knuth-Morris-Pratt [4] évite de revenir en arrière dans le texte grâce à la table `lps` : $lps[i]$ est la longueur du plus long préfixe propre de $P[0..i]$ qui est aussi un suffixe de $P[0..i]$. En cas d'échec après avoir reconnu j caractères, la comparaison reprend à la position $lps[j-1]$ du motif, sans relire le texte.

La table est calculée une seule fois par motif (`calculer_lps`), puis la fonction `recherche_kmp` parcourt chaque ligne et s'arrête à la première occurrence. Le choix entre KMP et l'automate est fait par `est_litteral` ; pour les mesures, le même motif littéral peut aussi être traité par l'automate (paramètre `methode` de `compiler`).

4 Analyse théorique et discussion

Soient r la taille du motif, n la longueur d'une ligne et $|\Sigma_R|$ le nombre de caractères distincts du motif (plus un pour `AUTRE`).

Étape	**Coût**	**Remarque**
Arbre syntaxique, AFN	$O(r)$ temps et espace	AFN de $O(r)$ états
Sous-ensembles	$O(2^r \cdot \lvert\Sigma_R\rvert \cdot r)$ au pire	le cas exponentiel n'apparaît pas sur nos motifs (tableau 2)
Minimisation (Moore)	$O(k^2\lvert\Sigma_R\rvert)$ pour un AFD à k états	Hopcroft : $O(k\lvert\Sigma_R\rvert \log k)$
Recherche par AFD	$O(n)$ par ligne	une transition par caractère
KMP	$O(m)$ prétraitement, $O(n)$ recherche	au plus $2n$ comparaisons

Table 3 – Complexité des différentes étapes.

Automate. Une fois l'AFD construit, le coût de la recherche est linéaire en la taille du texte et indépendant du motif. Le prix à payer est la déterminisation, exponentielle au pire : le motif $(\texttt{a|b})^*\texttt{a}(\texttt{a|b})^k$ par exemple donne un AFD de 2^{k+1} états. Deux alternatives existent dans la littérature : la simulation directe de l'AFN (ensemble d'états actifs mis à jour à chaque caractère, coût $O(nr)$ sans explosion) [7, 2], et la construction paresseuse de l'AFD, où seuls les sous-ensembles effectivement rencontrés dans le texte sont calculés, ce que font les implémentations efficaces comme GNU `grep` [2]. Pour les motifs courts considérés ici, la construction complète reste négligeable devant le parcours du texte. Les moteurs à retour arrière (*backtracking*), répandus dans les langages de script, peuvent quant à eux devenir exponentiels en la taille du texte [2] ; l'approche par automate garantit au contraire un parcours linéaire.

KMP. KMP revient à simuler l'automate des occurrences du motif sans le construire explicitement : le prétraitement est en $O(m)$ au lieu de $O(m|\Sigma|)$, et la recherche reste linéaire. Il n'existe donc pas de gain asymptotique par rapport à l'AFD minimal d'un motif littéral ; l'intérêt de KMP est d'éviter toute la chaîne de construction. En pratique, une transition d'AFD coûte une seule consultation de table, alors qu'une étape de KMP effectue comparaisons et mises à jour d'indices ; nous vérifions expérimentalement cet effet en section 6. Des algorithmes sous-linéaires en moyenne comme Boyer-Moore, utilisés par `grep` pour les motifs littéraux, permettent de sauter des portions du texte.

5 Tests de correction

5.1 Instances de test

Deux fichiers sont utilisés (répertoire `data/`) :

— `gutenberg_56667.txt` : le livre 56667 du Projet Gutenberg [5] (*A History of Babylon*, 13 309 lignes, 750 Ko), utilisé dans l'énoncé. Les fins de ligne `CRLF` ont été converties en `LF`, afin que la comparaison avec `grep` ne dépende pas de la gestion de `\r` par la plateforme ;

— `test_performance.txt` : un corpus synthétique de 500 000 lignes (35 Mo) produit par `scripts/generate_test_file.py` avec une graine fixe. Il contient toutes les 15 000 lignes une occurrence de `S(a|g|r)*on` (`Son`, `Sagaragon`, ...), toutes les 5 000 lignes un piège proche du motif (`Simon`, `Sgronx`, `Sargom`, ...) et du texte de remplissage ailleurs. Il n'est pas inclus dans l'archive mais régénéré à l'identique par `make data`.

5.2 Méthode et résultats

Le script `tests/validate.py` exécute le programme et `grep -E` (GNU grep 3.0) sur chaque couple (fichier, motif) et compare la sortie octet par octet ainsi que le code de retour. Les motifs couvrent les deux algorithmes, chaque opérateur, les étoiles imbriquées, le motif `.`, les motifs reconnaissant le mot vide (`a*`), un motif sans occurrence (`zzzz`) et les pièges du corpus synthétique. Les **22 tests réussissent** (tableau 4, `results/validation_results.csv`). Ces tests ne prouvent pas la correction pour tous les motifs, mais couvrent chaque construction prise en charge.

`gutenberg_56667.txt`			`gutenberg_56667.txt` (suite) et synthétique		
Motif	**Algo.**	**Lignes**	**Motif**	**Algo.**	**Lignes**
`end`	KMP	397	`B(a|e)*by`	AFD	1 178
`the`	KMP	7 952	`.`	AFD	11 593
`Sargon`	KMP	29	`a*`	AFD	13 309
`Babylon`	KMP	1 178	`x(y|z)*`	AFD	1 043
`zzzz`	KMP	0	*synthétique :*		
`a.b`	AFD	249	`S(a|g|r)*on`	AFD	49
`(the|and)`	AFD	8 874	`Simon`	KMP	11
`a.*e`	AFD	10 474	`Sargom`	KMP	15
`S(a|g|r)*on`	AFD	30	`Ligne 0`	KMP	1
`(a|b)*c`	AFD	8 535	`a.b`	AFD	0
`((ab)*|c)d`	AFD	9 469	`ab*`, `a|b`	AFD	500 000

TABLE 4 – Validation contre `grep -E` : nombre de lignes trouvées (sorties identiques pour les 22 couples fichier/motif).

L'exemple de l'énoncé (motif `S(a|g|r)+on`) utilise `+`, qui n'est pas pris en charge ; nous utilisons l'équivalent le plus proche `S(a|g|r)*on`, qui trouve 30 lignes du livre. Les premières lignes affichées sont les mêmes que dans la figure de l'énoncé (`results/example_output.txt`) :

```
$ python3 src/egrep_clone.py "S(a|g|r)*on" data/gutenberg_56667.txt
    state--Sargon and Merodach-baladan--Sennacherib's attempt
    under the Sargonids--The policies of encouragement and
that empire's expansion, and the vacillating policy of the Sargonids
to Sargon of Akkad; but that marked the extreme limit of Babylonian
...
```

6 Tests de performance

6.1 Protocole

Le script `scripts/benchmark.py` mesure, sur le livre Gutenberg, le temps de recherche de chaque motif : pour les quatre motifs littéraux, avec KMP, avec l'AFD minimal (même motif) et avec `grep -E` ; pour les cinq expressions régulières, avec l'AFD minimal et `grep -E`. Chaque mesure est précédée d'une exécution de chauffe puis répétée 10 fois ; les figures donnent la moyenne et l'écart-type (`results/benchmark_results.csv`). Pour le clone, le temps comprend la construction de l'automate (ou de la table `lps`), la lecture du fichier et la recherche, dans le processus Python ; pour `grep`, il comprend aussi le lancement du processus. Le nombre de lignes trouvées est vérifié identique pour toutes les méthodes.

Machine : Intel Core i5-1345U (10 cœurs), 16 Go de RAM, Windows 11, Python 3.13, GNU grep 3.0 (Git Bash).

6.2 Résultats

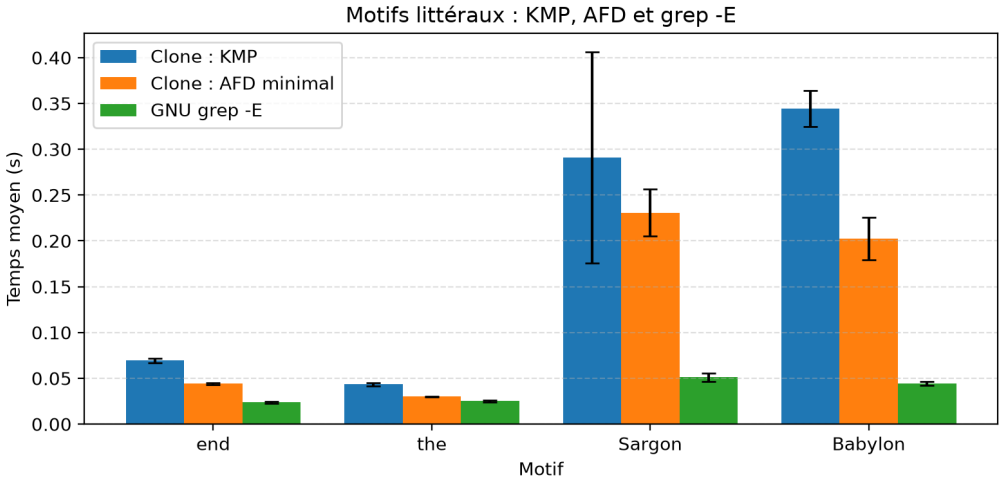


FIGURE 2 – Motifs littéraux : KMP, AFD minimal et `grep -E` (moyenne et écart-type sur 10 exécutions).

KMP contre automate (figure 2). Sur les quatre motifs littéraux, l'AFD minimal est plus rapide que KMP dans notre implémentation : par exemple 0,20 s contre 0,34 s pour `Babylon` et 0,044 s contre 0,069 s pour `end`. Comme prévu en section 4, les deux méthodes sont linéaires ; la différence vient du coût par caractère en Python (une consultation de dictionnaire pour l'AFD, plusieurs comparaisons et mises à jour d'indices pour KMP), le coût de construction de l'AFD étant négligeable pour un motif court. Le routage vers KMP n'apporte donc pas de gain de temps ici ; son avantage est d'éviter la déterminisation et de garantir un prétraitement en $O(m)$.

Clone contre `grep` (figures 2 et 3). `grep -E` est plus rapide pour tous les motifs : environ 0,02 à 0,05 s, dont environ 0,022 s de lancement du processus (mesuré sur un fichier vide), contre 0,03 à 0,23 s pour l'AFD (jusqu'à 5 fois plus lent). L'écart est le plus faible pour les motifs fréquents (`the`, `a.*e`), car l'arrêt à la première occurrence évite de lire la fin de la plupart des lignes ; il est maximal pour les motifs rares (`a.b`, `S(a|g|r)*on`), pour lesquels chaque ligne est lue entièrement. `grep` est écrit en C, travaille sur des blocs d'octets plutôt que ligne par ligne et utilise un AFD paresseux ainsi que Boyer-Moore pour sauter des portions du texte. Le temps du

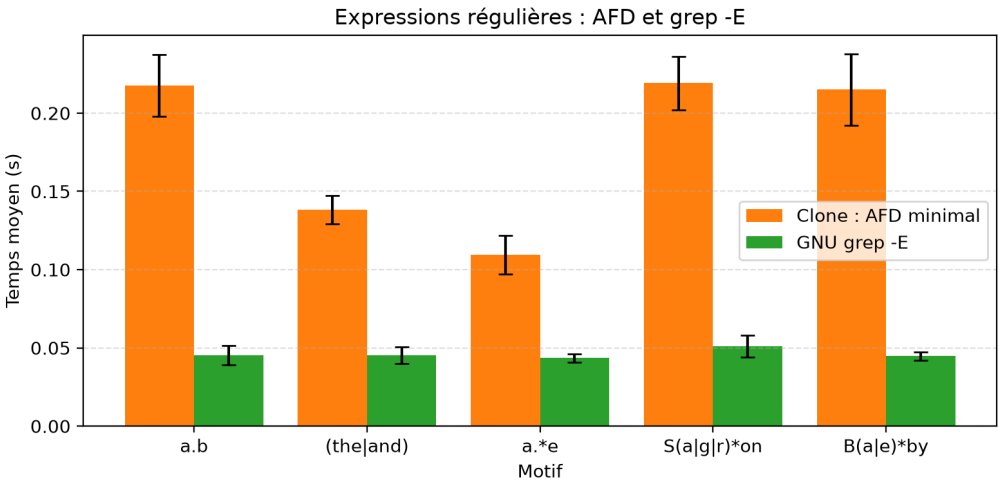


FIGURE 3 – Expressions régulières : AFD minimal et `grep -E` (moyenne et écart-type sur 10 exécutions).

clone dépend peu du motif à fréquence égale, ce qui est cohérent avec une recherche en $O(n)$ indépendante de la taille de l'automate.

7 Conclusion et perspectives

Nous avons réalisé un clone de `egrep` pour le sous-ensemble demandé des ERE, qui implémente la chaîne complète de la Séance 1 (arbre syntaxique, AFN d'Aho-Ullman, AFD par sous-ensembles, minimisation, recherche) et l'algorithme KMP de la Séance 2 pour les motifs littéraux. Ses sorties sont identiques à celles de `grep -E` sur les 22 cas testés. Les mesures montrent un temps de recherche linéaire, un AFD minimal plus rapide que KMP en Python, et un écart de performance avec `grep` dû principalement au langage et aux optimisations de bas niveau de `grep`.

Plusieurs pistes permettraient d'aller plus loin : compléter la norme ERE (`+`, `?`, classes de caractères, ancres, échappements) ; construire l'AFD paresseusement pour éviter l'explosion exponentielle ; utiliser un algorithme de type Boyer-Moore pour les littéraux et pour filtrer les lignes à l'aide d'un facteur obligatoire du motif. Enfin, dans le cas *offline* d'un fichier fixe interrogé de nombreuses fois, un précalcul d'une table d'index des mots du texte (*forward indexing*), stockée dans un fichier cache et interrogée via un trie, une table de hachage ou un filtre de Bloom, permettrait de ne parcourir que les lignes susceptibles de contenir le motif.

Références

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers : Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006. ISBN 9780321486813.

[2] Russ Cox. Regular expression matching can be simple and fast. `https://swtch.com/~rsc/regexp/regexp1.html`, January 2007. Consulté le 31 octobre 2026.

[3] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3 edition, 2006. ISBN 9780321455369.

[4] Donald E. Knuth, Jr. Morris, James H., and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2) :323–350, 1977. doi : 10.1137/0206024. URL `https://doi.org/10.1137/0206024`.

[5] Project Gutenberg. Project Gutenberg : Free ebooks. https://www.gutenberg.org/, 2026. Source de corpus textuels, consultée le 31 octobre 2026.

[6] The Open Group. The single UNIX specification, version 2 – regular expressions. http://pubs.opengroup.org/onlinepubs/7908799/xbd/re.html, 1997.

[7] Ken Thompson. Programming techniques : Regular expression search algorithm. *Communications of the ACM*, 11(6) :419–422, 1968. doi : 10.1145/363347.363387. URL https://doi.org/10.1145/363347.363387.